

1. Project Introduction

1.1 Project Title

CI/CD Pipeline Implementation for Automated Banking Application Testing using Parabank

1.2 Introduction

In today's software development environment, organizations continuously release new features, enhancements, and bug fixes to meet customer expectations and maintain competitiveness. Frequent software releases increase the need for efficient testing processes that ensure application quality without delaying deployment timelines. Manual testing alone becomes time-consuming, repetitive, and prone to human errors, especially when applications require repeated validation after every code change.

To overcome these challenges, organizations adopt **Test Automation** and **Continuous Integration/Continuous Deployment (CI/CD)** practices. Test automation helps execute repetitive test cases quickly and accurately, while CI/CD pipelines automate the build, testing, and deployment processes whenever changes are made to the application source code.

This project focuses on implementing a complete **CI/CD pipeline for automated testing of the Parabank Banking Application**. Parabank is a web-based banking application developed by Parasoft and is widely used for demonstrating automated testing concepts. The application provides various banking functionalities such as user registration, login, account overview, fund transfer, bill payment, loan requests, account management, and transaction tracking.

The primary objective of this project is to design and implement an automation framework using Selenium WebDriver and integrate it with GitHub Actions to achieve automated test execution whenever changes are pushed to the source code repository. This approach ensures early detection of defects, improves software quality, reduces manual effort, and supports faster software delivery.

1.3 Project Background

Software quality is one of the most critical factors for any banking application. Banking systems handle sensitive customer information, financial transactions, account management, and payment processing. Any defect within such systems may lead to incorrect transactions, financial losses, customer dissatisfaction, and security concerns.

Traditional manual testing approaches are not sufficient to support frequent releases and continuous development cycles. Therefore, automation testing combined with CI/CD practices plays a crucial role in maintaining software quality.

The Parabank application was selected for this project because it contains multiple real-world banking functionalities that can be automated and validated through different test scenarios. The project demonstrates how automated testing can be integrated into a CI/CD workflow to ensure application stability after every code change.

1.4 Project Objectives

The major objectives of this project are:

Primary Objectives

- Develop a robust Selenium automation framework using Java.
- Implement the Page Object Model (POM) design pattern.
- Create reusable and maintainable automation scripts.
- Execute data-driven testing using Excel files.
- Automate critical banking functionalities of the Parabank application.
- Integrate automation testing with GitHub Actions CI/CD pipeline.
- Generate automated test execution reports.
- Improve software quality through continuous testing.

Secondary Objectives

- Reduce manual testing effort.
 - Improve test execution speed.
 - Enable automated regression testing.
 - Detect defects at an early stage.
 - Demonstrate industry-standard QA automation practices.
-

1.5 Scope of the Project

The scope of this project includes:

Functional Testing

The following Parabank modules are covered:

1. User Registration
2. User Login
3. Account Overview
4. Open New Account

5. Transfer Funds
6. Bill Payment
7. Find Transactions
8. Request Loan
9. Update Contact Information

Automation Framework Development

The project includes:

- Selenium WebDriver automation
- Java programming
- TestNG framework
- Maven build management
- Page Object Model implementation
- Data-driven testing
- Screenshot capture on failure
- Test reporting

CI/CD Implementation

The project includes:

- Git version control
 - GitHub repository management
 - GitHub Actions workflow creation
 - Automated test execution on code push
 - Automated report generation
-

1.6 Technologies Used

Automation Tools

- Selenium WebDriver
- TestNG

Programming Language

- Java

Build Tool

- Maven

Design Pattern

- Page Object Model (POM)

Version Control

- Git

Repository Management

- GitHub

CI/CD Tool

- GitHub Actions

Test Data Management

- Apache POI
- Excel Files

Reporting

- TestNG Reports
- Extent Reports

IDE

- Eclipse IDE
-

1.7 Automation Framework Overview

The automation framework developed for this project follows a hybrid approach that combines:

Page Object Model (POM)

Each application page contains separate page classes that store web element locators and reusable methods.

Data-Driven Testing

Test data is stored in Excel files and read dynamically using Apache POI.

Reusable Components

Common functionalities such as browser initialization, configuration management, screenshot capture, and report generation are implemented as reusable utility components.

TestNG Integration

TestNG is used for:

- Test execution
 - Data providers
 - Assertions
 - Test grouping
 - Reporting
-

1.8 CI/CD Pipeline Overview

A GitHub Actions workflow is implemented to automate the testing process.

Whenever code is pushed to the GitHub repository:

1. Source code is checked out.
2. Java environment is configured.
3. Maven dependencies are downloaded.
4. Selenium test cases are executed.
5. Test reports are generated.
6. Reports are uploaded as workflow artifacts.

This automation eliminates manual execution efforts and provides immediate feedback regarding application quality.

1.9 Expected Outcomes

After successful completion of this project:

- Automated test scripts will validate major banking functionalities.
 - CI/CD pipeline will execute tests automatically.
 - Defects will be identified early in the development cycle.
 - Test reports will be generated automatically.
 - Regression testing efforts will be significantly reduced.
 - Software quality and release confidence will improve.
-

1.10 Benefits of the Project

The implementation of this project provides several benefits:

- Faster feedback on code changes
- Reduced manual testing effort
- Improved test coverage

- Consistent test execution
 - Early defect detection
 - Better software quality
 - Increased deployment confidence
 - Industry-standard automation practices
-

1.11 Conclusion

This project demonstrates the implementation of a complete test automation and CI/CD solution for a banking application using modern QA practices. By integrating Selenium automation with GitHub Actions, the project enables continuous testing and rapid feedback for every code change. The framework is scalable, maintainable, and aligned with industry standards, making it suitable for real-world software testing environments.

The successful implementation of this project showcases how automation testing and CI/CD pipelines can work together to improve software quality, reduce testing effort, and support continuous software delivery.